

```
❷ modified:   hello_git.py

❸ no changes added to commit (use "git add" and/or "git commit -a")
git_practice$
```

We see the branch we're working on ❶, the name of the file that was modified ❷, and that no changes have been committed ❸. Let's commit the change and check the status again:

```
❶ git_practice$ git commit -am "Extended greeting."
[main 945fa13] Extended greeting.
 1 file changed, 1 insertion(+), 1 deletion(-)
❷ git_practice$ git status
On branch main
nothing to commit, working tree clean
❸ git_practice$ git log --pretty=oneline
945fa13af128a266d0114eebb7a3276f7d58ecd2 (HEAD -> main) Extended greeting.
cea13ddc51b885d05a410201a54faf20e0d2e246 Started project.
git_practice$
```

We make a new commit, passing the `-am` flags when we use the command `git commit` ❶. The `-a` flag tells Git to add all modified files in the repository to the current commit. (If you create any new files between commits, reissue the `git add` . command to include the new files in the repository.) The `-m` flag tells Git to record a message in the log for this commit.

When we check the project's status, we see that we once again have a clean working tree ❷. Finally, we see the two commits in the log ❸.

Abandoning Changes

Now let's look at how to abandon a change and go back to the previous working state. First, add a new line to *hello_git.py*:

```
hello_git.py print("Hello Git world!")
              print("Hello everyone.")

              print("Oh no, I broke the project!")
```

Save and run this file.

We check the status and see that Git notices this change:

```
git_practice$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)

❶   modified:   hello_git.py

no changes added to commit (use "git add" and/or "git commit -a")
git_practice$
```
